

Architecture serveurs & sécurité — Concorde Workforce

Référence : RAPPORT_ARCHITECTURE_INFRA — version 1.0 — 2026-05-26 **Périmètre :** infrastructure de production de la plateforme SaaS multi-tenant Concorde Workforce (pointage, gestion du temps, RH).
Auteur : Équipe technique — Concorde Tech Innovation.

1. Objet du document

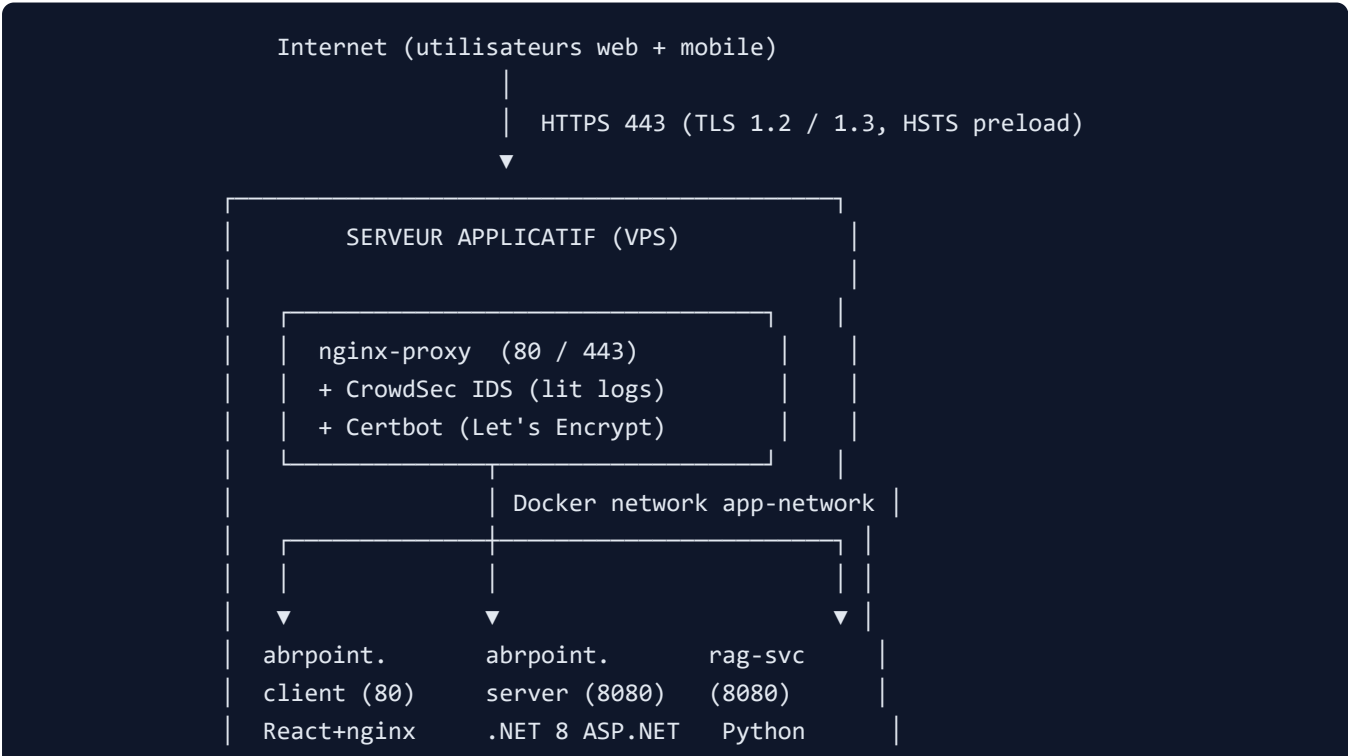
Ce rapport décrit l'architecture des serveurs en production, les flux de communication entre composants, et les mesures de sécurité opérationnelles mises en place. Il s'adresse à un public technique (DSI, DPO, auditeur sécurité, équipe ops) et complète la checklist `SECURITY_INFRA_CHECKLIST.md`.

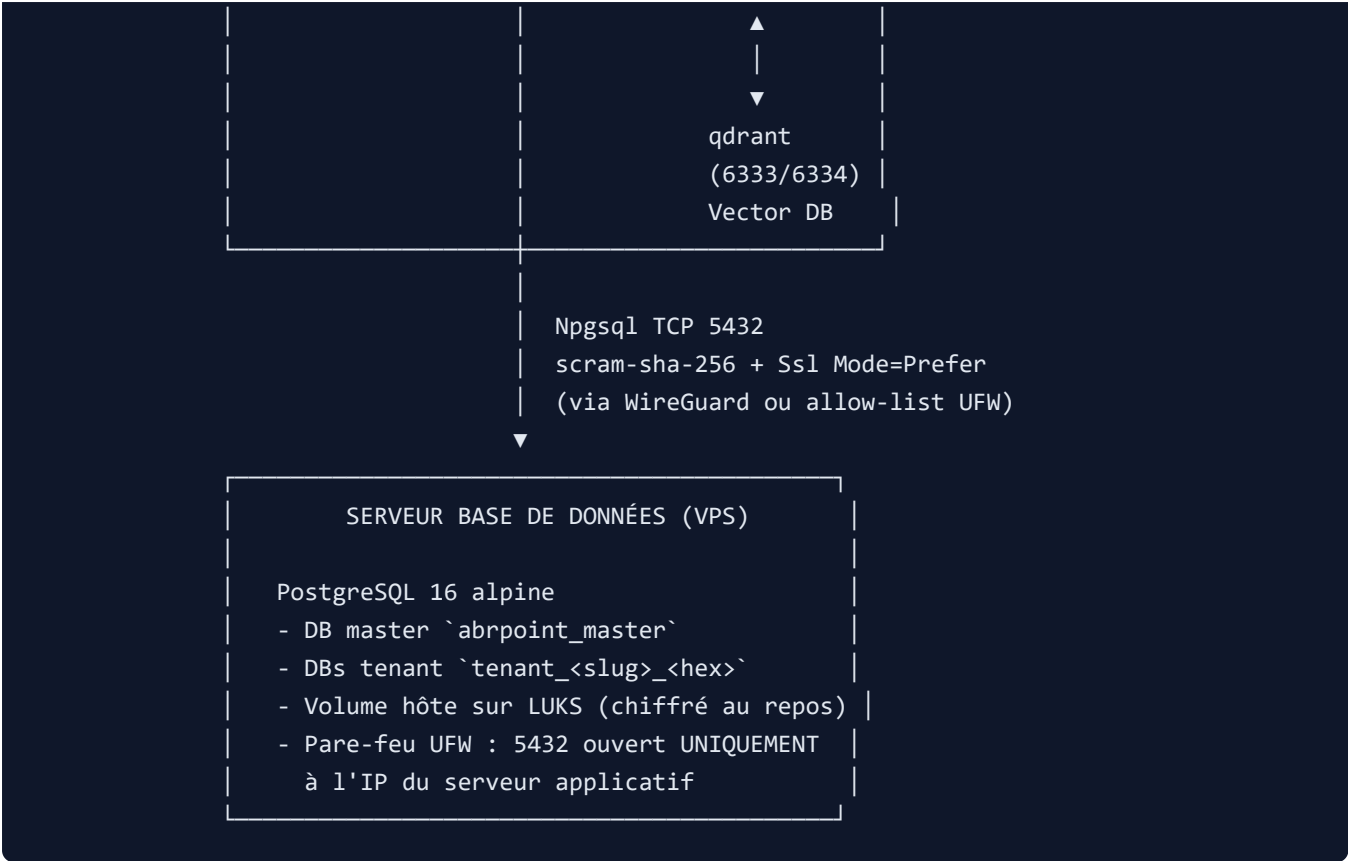
La plateforme repose sur **deux serveurs distincts**, déployés via Docker Compose :

- un **serveur dédié à la base de données** (PostgreSQL 16 standalone) ;
- un **serveur applicatif** hébergeant le backend .NET, le frontend React, le moteur de recherche vectorielle Qdrant, le micro-service RAG, le reverse-proxy nginx et le système de détection d'intrusion CrowdSec.

Cette séparation physique répond à une exigence de défense en profondeur (Art. 32 RGPD) : la base contenant l'intégralité des données salariés est isolée du serveur exposé à Internet.

2. Vue d'ensemble





Modèle multi-tenant : chaque société cliente dispose de sa propre base PostgreSQL (`tenant_<slug>_<hex>`), provisionnée dynamiquement par le backend à l'inscription. Une base maître (`abrpoint_master`) centralise les métadonnées tenants (statut, plan, clés Stripe, addons souscrits) et l'index global email→slug pour le routage du login.

3. Serveur de base de données

3.1 Composant

Caractéristique	Valeur
Image	postgres:16-alpine
Port d'écoute	5432/tcp
Authentification	scram-sha-256 (forcée via POSTGRES_INITDB_ARGS)
Volume de données	/var/lib/abrpoint/postgres (hôte)
Volume de backups	postgres_backup (Docker named volume)
Connexions max	300
Shared buffers	512 Mo
Logs requêtes lentes	seuil 500 ms
Healthcheck	pg_isready toutes les 10 s

Cf. `docker-compose.db.yml` .

3.2 Bases hébergées

- `abrpoint_master` — créée au premier démarrage. Tables clés : `Tenants` , `TenantEmailIndex` , `RefreshTokens` (révocables). Le superuser `abrpoint` conserve le rôle `CREATEDB` indispensable au provisioning dynamique.
- `tenant_{slug}_{8 chars hex}` — créée à chaque inscription via `ProvisioningService` côté backend. Schéma EF Core migré automatiquement (Code-First). Tables métier : `Utilisateurs` , `Employes` , `Presences` , `Conges` , `Autoriser` , `audit_log` , etc.

3.3 Isolation

Le serveur DB n'héberge **aucun service exposé à Internet** : pas de SSH externe, pas de panel d'admin. Seul le port `5432` est ouvert, restreint au serveur applicatif (cf. §7.1).

4. Serveur applicatif

4.1 Conteneurs

Six services orchestrés via `docker-compose.app.yml` , tous connectés au réseau Docker interne `app-network` :

Conteneur	Image	Port interne	Rôle
<code>nginx-proxy</code>	<code>nginx:alpine</code>	80 / 443 (publics)	Reverse-proxy TLS, terminaison HTTPS, en-têtes de sécurité, routage <code>/api</code> vs SPA
<code>abrpoint.client</code>	<code>mohamedbenrhaiem2003/abrpoint-client</code>	80	SPA React+Vite servie statiquement par un nginx interne
<code>abrpoint.server</code>	<code>mohamedbenrhaiem2003/abrpoint-server</code>	8080	API REST .NET 8 (ASP.NET Core, EF Core, Npgsql)
<code>rag-svc</code>	<code>mohamedbenrhaiem2003/abrpoint-rag-svc</code>	8080	Side-car Python d'embeddings (multilingual-e5-large)
<code>qdrant</code>	<code>qdrant/qdrant:v1.12.0</code>	6333 / 6334	Base vectorielle (recherche sémantique RAG)
<code>crowdsec</code>	<code>crowdsecurity/crowdsec:latest</code>	(aucun port exposé)	IDS lisant les logs nginx, génère des décisions de blocage

4.2 Reverse-proxy nginx

Le conteneur `nginx-proxy` est l'**unique point d'entrée HTTPS** du serveur applicatif :

- Écoute `80` → redirection 301 vers `https://` (sauf `/.well-known/acme-challenge/` pour Let's Encrypt).
- Écoute `443` → certificats Let's Encrypt (renouvelés par `certbot`), `ssl_protocols TLSv1.2 TLSv1.3` .

- Routage interne : `location / → upstream frontend = abrpont.client:80 ; location /api → upstream backend = abrpont.server:8080 .`
- Rate-limit applicatif : `limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s` sur `/api` .
- En-têtes ajoutées systématiquement : `Strict-Transport-Security` (max-age 1 an + preload), `X-Content-Type-Options nosniff` , `X-Frame-Options SAMEORIGIN` , `Referrer-Policy strict-origin-when-cross-origin` , `Content-Security-Policy restrictive` (sources allow-listées).
- `server_tokens off` : la version nginx n'est jamais exposée dans les headers.

Cf. `nginx.conf` .

4.3 Backend applicatif

Le conteneur `abrpont.server` exécute l'API .NET 8 :

- **Authentification** : JWT signés HS256 (clé `Jwt__Key` , ≥ 32 caractères aléatoires), refresh tokens hashés SHA-256 stockés en master DB.
- **Multi-tenant** : middleware `TenantResolverMiddleware` lit le claim `tenant_slug` du JWT ou le header `X-Tenant-Slug` , résout la base cible et injecte la chaîne de connexion dans le scope de la requête.
- **Chiffrement applicatif** : `EncryptionService` (AES-256-GCM, clé `Encryption__AesKey`) pseudonymise les colonnes ultra-sensibles (`Employe.Empcin` , `Empsbase` , `Empbrut` , `Empnet` , `Emptel`) via un EF Core value converter — aucun nouveau code ne peut écrire ces champs en clair.
- **MFA TOTP** : optionnel par utilisateur, fondé sur la lib `Otp.NET` .
- **HIBP** : `PasswordBreachCheckService` interroge l'API Have-I-Been-Pwned en k-anonymity (préfixe SHA-1 sur 5 caractères) pour refuser les mots de passe figurant dans des fuites.
- **Rate-limit + lockout** : politique `[EnableRateLimiting("auth-signup")]` sur le signup (3/h/IP), verrouillage progressif après échecs de login.

4.4 Frontend applicatif

Le conteneur `abrpont.client` sert la SPA React build (Vite) via un nginx léger. Aucun rendu côté serveur, donc surface d'attaque minimale (pas de SSRF possible côté front).

4.5 Pipeline RAG (IA documentaire)

- `rag-svc` : Python (FastAPI) — charge `intfloat/multilingual-e5-large` au démarrage, expose `/embed` et `/query` protégés par un `SIDECAR_KEY` partagé avec le backend.
- `qdrant` : stocke les embeddings (un namespace par tenant via le préfixe de collection), expose 6333 (HTTP) et 6334 (gRPC) **uniquement sur le réseau Docker** — jamais publié.

4.6 IDS / IPS — CrowdSec

`crowdsec` lit en lecture seule les logs nginx (volume partagé `nginx_logs`) et déroule trois collections :

- `crowdsecurity/nginx` — brute-force d'authentification, scan de paths.
- `crowdsecurity/http-cve` — patterns d'exploitation de CVE connues.
- `crowdsecurity/base-http-scenarios` — OWASP top 10 génériques (path traversal, injection, scrapping massif).

Le partage des indicateurs avec la base communautaire est désactivé (`DISABLE_ONLINE_API: true`) en attente d'une décision DPO. Le **bouncer** (composant qui matérialise le blocage) est documenté dans la checklist (`firewall-bouncer-iptables` côté host).

5. Communication entre le serveur applicatif et le serveur de base de données

5.1 Protocole

Le backend `abrpoint.server` ouvre des connexions PostgreSQL via **Npgsql** (driver .NET officiel) sur le port `5432` du serveur DB. Les chaînes de connexion sont injectées au démarrage :

```
ConnectionStrings__MasterConnection: Host=${DB_HOST};Port=5432;Database=abrpoint_master;
Username=abrpoint;Password=${POSTGRES_PASSWORD};
Ssl Mode=Prefer;Trust Server Certificate=true;
Pooling=true;Maximum Pool Size=100
```

Paramètre	Effet sécurité
<code>Ssl Mode=Prefer</code>	Le client tente TLS ; retombe en clair uniquement si le serveur ne le supporte pas. À durcir en <code>Require</code> dès que le serveur DB exposera un certificat (cf. §7.2).
<code>scram-sha-256</code> côté serveur	Le mot de passe ne transite jamais en clair (challenge-response salé).
<code>Maximum Pool Size=100</code>	Limite la consommation de connexions par instance backend (protection contre DoS interne).
<code>Pooling=true</code>	Réutilisation des connexions ouvertes (perf + moins de handshake TLS).

5.2 Canaux de transport recommandés

Trois options, du plus sûr au plus simple à mettre en œuvre :

1. **VPN site-à-site WireGuard (recommandé)** — un tunnel chiffré UDP entre les deux VPS. Le port `5432` n'est jamais exposé sur l'IP publique du serveur DB ; seul le pair WireGuard de l'app est joignable. Aucune exposition Internet.
2. **VPC privé du fournisseur cloud (équivalent)** — deux VPS dans le même VPC OVH/Scaleway/AWS, communication via IPs privées.
3. **IP whitelisting via pare-feu UFW** — fallback si VPN indisponible. Le port `5432` reste ouvert sur Internet, mais filtré au niveau OS (cf. §7.1).

Dans tous les cas, l'option `Ssl Mode=Prefer` doit être promue en `Ssl Mode=Require` avec certificats serveur Postgres dès qu'un certificat fiable est disponible.

5.3 Création dynamique de bases tenant

Lors d'un signup, `ProvisioningService` exécute, **côté backend uniquement** :

1. `CREATE DATABASE tenant_<slug>_<hex>` (le rôle `abrpoint` a `CREATEDB`).
2. Application des migrations EF Core via `dotnet ef` -équivalent embarqué.
3. Seed de la société, du site principal et de l'admin `AD` avec mot de passe BCrypt et code OTP de vérification email.

Le superuser DB n'est jamais utilisé directement par le code applicatif : tout passe par le rôle `abrpoint` .

6. Communication interne au serveur applicatif

Tous les conteneurs sont membres du réseau Docker bridge `app-network` — réseau privé, non routé vers l'extérieur. Les flux internes sont les suivants :

Source	Destination	Port	Protocole	Authentification
<code>nginx-proxy</code>	<code>abrpoint.client</code>	80	HTTP	— (frontend public)
<code>nginx-proxy</code>	<code>abrpoint.server</code>	8080	HTTP	JWT (cookie ou header <code>Authorization</code>)
<code>abrpoint.server</code>	<code>rag-svc</code>	8080	HTTP	Clé partagée <code>SIDECAR_KEY</code>
<code>rag-svc</code>	<code>qdrant</code>	6333	HTTP	— (réseau privé, isolé)
<code>crowdsec</code>	<code>nginx_logs</code> (volume)	—	Lecture seule	Volume Docker <code>:ro</code>
<code>certbot</code>	<code>letsencrypt_data</code> (volume)	—	Lecture/écriture	Partagé avec <code>nginx-proxy</code> en lecture seule

Principes d'isolation :

- Aucun port interne n'est publié sur l'IP de l'hôte. `nginx-proxy` est le seul service à mapper `80` et `443` vers l'extérieur.
- Les conteneurs ne se voient que par leur nom DNS Docker (`abrpoint.server` , `qdrant` , etc.), pas par adresse IP.
- Le volume `nginx_logs` est monté en `:ro` côté CrowdSec — l'IDS ne peut pas altérer les logs nginx.

7. Mesures de sécurité

7.1 Périmètre réseau (pare-feu)

Sur le serveur DB :

```
ufw default deny incoming
ufw allow ssh # ou port custom + clé SSH
ufw allow from <APP_SERVER_IP> to any port 5432 proto tcp
ufw enable
```

Le port `5432` n'est joignable qu'à partir de l'IP publique du serveur applicatif. Toute autre tentative est silencieusement rejetée. La règle est complétée par un audit `ufw status numbered` archivé après chaque modification.

Sur le serveur applicatif :

```
ufw default deny incoming
ufw allow ssh
ufw allow 80/tcp
ufw allow 443/tcp
ufw enable
```

Seuls les ports HTTP/HTTPS sont publics. Tous les autres conteneurs (backend `:8080`, qdrant `:6333/6334`, rag-svc `:8080`) ne sont accessibles qu'au sein du réseau Docker `app-network`.

7.2 Chiffrement

En transit :

- HTTPS public : Let's Encrypt, TLS 1.2 et 1.3 uniquement, HSTS `max-age=31536000; preload`, suite de chiffrement par défaut nginx stricte.
- Backend ↔ DB : `Ssl Mode=Prefer` (à durcir en `Require` dès certificats disponibles).
- Mobile : **certificate pinning** sur les certificats Let's Encrypt intermédiaires (`network_security_config.xml` Android, `NSPinnedDomains` iOS). Les pins sont régénérés annuellement et vérifiés en pré-build EAS (`check-pins-filled.sh`).

Au repos :

- Volumes PostgreSQL chiffrés via **LUKS** au niveau OS (`cryptsetup luksFormat /dev/sdX`). Clé conservée hors-machine (coffre Bitwarden ops), montée au boot via `/etc/crypttab`.
- Pseudonymisation applicative AES-256-GCM des champs sensibles (CIN, téléphone, salaires) — clé `Encryption__AesKey` injectée par variable d'environnement, jamais committée.
- Mots de passe utilisateurs : **BCrypt** (cost factor 12 par défaut, recalibré périodiquement).
- Refresh tokens : hash **SHA-256** (les tokens en clair ne sont stockés qu'en cookie côté client, jamais en base).

7.3 Détection d'intrusion (CrowdSec)

Élément	Détail
Source de signal	Logs nginx (<code>access.log</code> , <code>error.log</code>) en lecture seule
Scénarios actifs	nginx (brute-force, scan), <code>http-cve</code> , <code>base-http-scenarios</code> (OWASP)
Décisions	Stockées localement (<code>crowdsec_data</code> volume)
Blocage effectif	Bouncer host (<code>crowdsec-firewall-bouncer-iptables</code>) — étape ops documentée dans <code>SECURITY_INFRA_CHECKLIST.md §4.b</code>
Community blocklist	Désactivée (<code>DISABLE_ONLINE_API: true</code>) en attente décision DPO

7.4 Sécurité applicative

Authentification :

- JWT HS256, durée de vie courte (1 h), rotation via refresh tokens.
- Cookies `HttpOnly` , `Secure` , `SameSite=None` (en HTTPS) ou `Lax` (en dev local).
- MFA TOTP optionnel.
- Verrouillage progressif après échecs de login répétés.
- Vérification email obligatoire au signup (code OTP 6 chiffres, durée 15 min).
- HIBP k-anonymity au signup et au changement de mot de passe.

Autorisation :

- RBAC explicite (`PermissionCatalog` , table `RolePermission`).
- Gating commercial par feature flag : attribut `[RequirePlanFeature(...)]` sur les contrôleurs (26 endpoints protégés). Cf. matrice `PlanCatalog` .

Anti-CSRF / SSRF :

- Cookies `SameSite=None` couplés à `Secure` (CSRF mitigated en HTTPS uniquement).
- Pas d'appel sortant configurable par l'utilisateur côté backend (pas de SSRF).
- CSP nginx restrictive : `default-src 'self'` , sources allow-listées (`restcountries.com` , `calendrier.api.gouv.fr`).

Captcha & anti-bot :

- Captcha arithmétique single-use sur le signup, validation côté serveur.
- Rate-limit signup : 3 inscriptions / heure / IP (politique `auth-signup`).

7.5 Sécurité mobile

- Certificate pinning Let's Encrypt (renouvellement annuel + alerte 60 j avant expiration).
- Détection device : heuristiques étendues (`deviceSecurity.ts`) — émulateur, jailbreak, debug build, instrumentation Frida/objection.
- Trust report envoyé au backend (`/api/MobileAuth/device-trust-report`), journalisé 6 mois.
- Protection capture d'écran (`useSecureScreen`) sur les écrans sensibles (signature, salaires).

7.6 Journalisation & rétention

- `audit_log` : toute action sensible (login, change-plan, suppression). Rétention 6 mois via `AuditLogRetentionHostedService` .
- `DataRetentionHostedService` : purge automatique des refresh tokens expirés, devices inactifs, notifications push obsolètes, embeddings RAG des tenants résiliés.
- Logs nginx rotés (logrotate hôte) + envoyés à CrowdSec.
- Plan futur : envoi vers SIEM (Wazuh / Graylog) sur incident critique.

7.7 Patching & veille CVE

Mécanisme	Cible	Fréquence	Bloquant ?
<code>unattended-upgrades</code> Ubuntu	OS hôte (canal <code>security</code>)	Quotidien	—
Dependabot (Docker, NuGet, npm)	Images de base + dépendances	Hebdomadaire	PR auto
Trivy filesystem scan	Secrets + misconfigs Dockerfile	À chaque PR + hebdo	✅ HIGH/CRITICAL
Trivy image scan	CVE des images buildées	À chaque PR + hebdo	Informationnel
<code>dotnet list package --vulnerable</code>	Packages NuGet	À chaque PR	✅
<code>npm audit --audit-level=high</code>	npm client + mobile	À chaque PR	✅
<code>pip-audit</code>	Dépendances Python sidecar RAG	À chaque PR	⚠️ Informationnel

7.8 Sauvegardes

- Dumps PostgreSQL chiffrés (`openssl enc -aes-256-cbc`) poussés vers un bucket **S3 EU** (eu-west-3 Paris ou eu-central-1 Frankfurt), versioning + Object Lock activés.
- Lifecycle rule : transition Glacier Instant Retrieval à J+30, expiration J+365.
- Test de restauration mensuel (`scripts/restore-test.sh`), alerte email en cas d'échec.
- Clé de chiffrement `BACKUP_ENC_KEY` conservée hors-bucket (coffre ops Bitwarden), injection via variables d'environnement.

8. Conformité RGPD Art. 32 — Récapitulatif

Exigence	Implémentation	Référence code/infra
TLS 1.2+	✅ Validé	<code>nginx.conf</code> (TLSv1.2/1.3, HSTS preload)
Chiffrement au repos	✅ Validé	<code>SECURITY_INFRA_CHECKLIST.md §1</code> (LUKS sur volume Postgres)
Pseudonymisation PII	✅ Validé	<code>Data/EncryptedStringConverter.cs</code> , <code>EncryptionService.cs</code>
BCrypt mots de passe	✅ Validé	<code>Utilisateur.Utimes</code>
Refresh tokens hashés	✅ Validé	<code>Services/RefreshTokenHasher.cs</code>
MFA TOTP	✅ Validé	<code>Otp.NET</code> + <code>UtilisateursController</code>
HIBP k-anonymity	✅ Validé	<code>Services/PasswordBreachCheckService.cs</code>
Verrouillage progressif	✅ Validé	<code>Utilisateur</code> + rate limiter ASP.NET

Exigence	Implémentation	Référence code/infra
Alerte nouvel appareil	✓ Validé	Services/SuspiciousLoginTokenService.cs
Mobile : pinning / device check / screenshot block	✓ Validé	deviceSecurity.ts , network_security_config.xml , useSecureScreen.ts
Cloisonnement environnements	✓ Validé	SECURITY_INFRA_CHECKLIST.md §5
Audit logs + rétention 6 mois	✓ Validé	Services/AuditLogRetentionHostedService.cs
Purge données techniques	✓ Validé	Services/DataRetentionHostedService.cs
Patching dépendances + images	✓ Validé	.github/dependabot.yml , .github/workflows/security-scan.yml
Patching OS (Ubuntu)	✓ Validé	scripts/setup-unattended-upgrades.sh
Sauvegardes chiffrées S3 EU	✓ Validé	scripts/backup.sh , scripts/restore-test.sh
Anti-malware / IDS	✓ Validé	docker-compose.yml (service crowdsec) + bouncer firewall actif
Firewall	✓ Validé	UFW (cf. §7.1 et §9.1)

Légende : ✓ implémenté et validé en production.

9. Procédure de déploiement (résumé)

9.1 Serveur DB

```
# Préparation volume LUKS
sudo cryptsetup luksFormat /dev/sdb
sudo cryptsetup luksOpen /dev/sdb postgres_data
sudo mkfs.ext4 /dev/mapper/postgres_data
sudo mkdir -p /var/lib/abrpoin/postgres
sudo mount /dev/mapper/postgres_data /var/lib/abrpoin/postgres

# Pare-feu
sudo ufw default deny incoming
sudo ufw allow ssh
sudo ufw allow from <APP_SERVER_IP> to any port 5432 proto tcp
sudo ufw enable

# Démarrage Postgres
export POSTGRES_PASSWORD=<mot-de-passe-fort-32-caractères>
docker compose -f docker-compose.db.yml up -d
```

9.2 Serveur applicatif

```
# Pare-feu
sudo ufw default deny incoming
sudo ufw allow ssh
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable

# Variables d'environnement (cf. .env.app.example)
cat > .env.app <<EOF
DB_HOST=<IP_VPN_ou_publique_serveur_DB>
DB_PORT=5432
POSTGRES_PASSWORD=<même mot de passe que serveur DB>
JWT_SECRET=<openssl rand -base64 48>
AES_KEY=<openssl rand -base64 32>
ANTHROPIC_API_KEY=<...>
OPENROUTER_API_KEY=<...>
STRIPE_SECRET_KEY=<...>
STRIPE_WEBHOOK_SECRET=<...>
GEMINI_API_KEY=<...>
SMTP_HOST=ssl0.ovh.net
SMTP_PORT=587
SMTP_USERNAME=<...>
SMTP_PASSWORD=<...>
SMTP_FROM_EMAIL=<...>
RAG_SIDEKAR_KEY=<openssl rand -base64 32>
ADMIN_BOOTSTRAP_PASSWORD=<mot de passe initial admin>
EOF

# Bootstrap Let's Encrypt (une seule fois)
docker compose -f docker-compose.app.yml run --rm certbot certonly --webroot \
  -w /var/www/certbot -d concorde-work-force.com -d www.concorde-work-force.com \
  --email mohamedberhaem43@gmail.com --agree-tos --non-interactive

# Démarrage de la stack
docker compose -f docker-compose.app.yml --env-file .env.app up -d

# CrowdSec bouncer (post go-live)
sudo apt install crowdsec-firewall-bouncer-iptables
docker exec abrpoint.crowdsec cscli bouncers add fw-bouncer
# Coller la clé dans /etc/crowdsec/bouncers/crowdsec-firewall-bouncer.yaml
sudo systemctl restart crowdsec-firewall-bouncer
```

10. Annexes

- `SECURITY_INFRA_CHECKLIST.md` — checklist détaillée des mesures ops (chiffrement, sauvegardes, IDS, patching).

- `docker-compose.app.yml` — stack serveur applicatif.
 - `docker-compose.db.yml` — stack serveur DB.
 - `nginx.conf` — configuration reverse-proxy TLS.
 - `scripts/backup.sh` — sauvegardes chiffrées vers S3 EU.
 - `scripts/restore-test.sh` — test mensuel de restauration.
 - `scripts/setup-unattended-upgrades.sh` — patching automatique Ubuntu.
-

Document maintenu par l'équipe technique Concorde Tech Innovation. Toute modification doit être tracée via une PR sur le dépôt principal et mentionnée dans le changelog du projet.